

CSCE 753 COMPUTER VISION · MODULE 11 · LECTURE NOTES

Neural Scene Representations

Inverse rendering, finally.

MODULE OUTCOMES

- Explain a radiance field and how it is rendered.
- Explain why the rendering must be differentiable.
- Explain 3D Gaussian splatting and why it is faster.
- Compare these against classical multi-view stereo honestly.
- State what these representations are and are not suitable for.

1 Inverting rendering by differentiating it

INVERT RENDERING BY DIFFERENTIATING IT

Classical reconstruction searches for correspondences and then triangulates (Modules 04 through 06). **Correspondence is the hard and brittle part** — Module 06 §4's entire failure list is correspondence failure.

Neural reconstruction skips it. Parameterise the scene somehow, *render* that parameterisation from each known camera pose, and **minimise the difference between the rendered image and the captured photograph**. No correspondence is ever computed or needed.

If the renderer is differentiable, gradient descent solves it — which is CSCE 647's forward model used as a loss function, with CSCE 669 Module 05's first-order methods as the optimiser. Both of those courses were prerequisites for exactly this.

The requirement is that rendering be differentiable with respect to the scene parameters, and rasterisation is not: move a triangle slightly and a pixel's visible surface can switch discontinuously, so the derivative does not exist. **Resolving that is the technical core of the module** (§2), and it is what determined the representation.

$$\mathbf{F}(\mathbf{x}, \mathbf{y}, \mathbf{z}, \theta, \phi) \rightarrow (\mathbf{r}, \mathbf{g}, \mathbf{b}, \sigma)$$

$$\mathbf{C}(\text{ray}) = \int \mathbf{T}(\mathbf{t}) \cdot \sigma(\mathbf{t}) \cdot \mathbf{c}(\mathbf{t}) \, d\mathbf{t}, \quad \mathbf{T}(\mathbf{t}) = \exp\left(-\int \sigma(\mathbf{s}) \, d\mathbf{s}\right)$$

A radiance field is a function from position and viewing direction to colour and density, stored as a small multilayer perceptron, a voxel grid, or a multiresolution hash table. **Rendering is volume rendering:** integrate emitted colour along the ray, weighted by local density and by the transmittance $\mathbf{T}$ — the probability that light reaches that point unoccluded. **This is precisely CSCE 647's**

participating-media integral, which is why that course's treatment of volumetric transport was worth the effort it took.

2 Why a volume

SURFACES ARE NOT DIFFERENTIABLE; VOLUMES ARE

Move a triangle slightly and a pixel's visible surface may switch discontinuously from one triangle to another. The derivative of the image with respect to the geometry does not exist at that configuration, and gradient descent cannot cross it.

A density field has no such discontinuity. Every point along the ray contributes continuously, weighted by transmittance, so **the rendered colour is a smooth function of the scene parameters everywhere** — which is exactly what the optimiser requires.

So the volumetric representation was chosen for optimisability rather than for physical accuracy. **Real scenes are overwhelmingly surfaces**, and representing a wooden table as a cloud of dense fog is a deliberate concession to the optimiser — one worth understanding, because it explains the method's characteristic artefacts.

Which is why extracting a clean mesh from a radiance field is difficult — there is no surface in the representation, only a density level set that may be thick, noisy, or ambiguous — and why signed-distance-function variants (VolSDF, NeuS) exist: **they reintroduce an explicit surface while keeping a smooth gradient path to it**, which is CSCE 645 Module 04's implicit surfaces used for a new purpose.

POSITIONAL ENCODING IS NOT OPTIONAL. An MLP applied to raw (x,y,z) produces a blurry field -- it cannot represent high spatial frequencies. Encode each coordinate as $\sin/\cos$ at many frequencies first, then feed that.

This is CSCE 647's sampling theory: the network's capacity to represent detail is bounded by the frequency content of its input, and raw coordinates have none.

HIERARCHICAL SAMPLING. Most of a ray is empty space. Sample coarsely, build a density-weighted distribution along the ray, then sample finely where the density actually is. A 2-3x speedup for no accuracy cost, and it is importance sampling (CSCE 647 Module 03).

CAMERA POSES MUST BE ACCURATE. NeRF assumes them known, and in practice they come from COLMAP (Module 05). A pose error of one degree produces visible blur that looks exactly like under-training -- which sends people to the wrong fix and wastes days.

WHAT GOES WRONG

too few views	-> FLOATERS: density in empty space that happens to fit every input image from the angles you captured
all views one side	-> the unseen side is INVENTED, with no indication that it was
moving objects	-> smeared, exactly as in Module 05
changing exposure	-> baked into the radiance field as if it were scene appearance
mirrors	-> modelled as GEOMETRY BEHIND the mirror, which is self-consistent with every input view and completely wrong

Every one of those failures is Module 01's ill-posedness, arriving in a new costume. The optimiser found a scene consistent with the photographs that is not the real scene — which is permitted, because the inversion is under-determined and nothing in the objective prefers the true explanation. **The mirror case is the most illuminating: a room behind the glass explains every input view correctly**, it is geometrically reasonable, and it is physically wrong — and no amount of additional views from the same side of the mirror resolves it. **Resolving it needs a prior about materials**, which is Module 01 §3's fourth strategy and is what reflection-aware variants supply.

3 Gaussian splatting

	NeRF (radiance field)	3D Gaussian splatting
Scene stored as	An implicit function — an MLP, grid, or hash table.	An explicit set of 3D Gaussians, each with position, covariance, opacity, and view-dependent colour.
Rendering	Ray marching, with many network queries per pixel.	Project each Gaussian to 2D, rasterise, and alpha-blend sorted by depth.
Render speed	Seconds per frame in the original method.	Real time — hundreds of frames per second.
Training time	Hours.	Minutes.
Editability	Hard — the network weights are not spatially local, so editing one region affects others.	Easy — move, delete, or recolour individual primitives.
Memory	Small — a network of a few megabytes.	Large — millions of primitives, hundreds of megabytes.

Splatting won on speed and editability by going back to an explicit representation, after several years in which implicit neural representations were assumed to be the direction. **That is a useful reminder that implicit is not automatically better** — the implicit formulation was solving a differentiability problem, and once that problem could be solved explicitly, the explicit form's advantages reasserted themselves.

WHY SPLATTING IS FAST, EXACTLY

A 3D Gaussian projects to a 2D Gaussian under perspective projection (to a good approximation), **and a 2D Gaussian can be rasterised** — so the entire scene renders through the graphics pipeline of CSCE 641 rather than by marching rays through a field.

There is no network query per sample. NeRF evaluates an MLP hundreds of times per pixel; splatting alpha-blends a few dozen primitives per pixel, with no neural network in the rendering loop at all.

And alpha blending sorted by depth is differentiable, which recovers §2's essential property with an explicit primitive — **because a Gaussian has no hard boundary**, so its contribution to a pixel varies smoothly as it moves. The discontinuity that defeated triangles is absent.

The adaptive density control is the genuinely clever part. Split Gaussians where the positional gradient is large (the representation is under-resolved there) and prune ones that become transparent, so the representation adapts its own resolution to the scene during optimisation — which is what replaces NeRF's hierarchical sampling and is why no resolution needs to be chosen in advance.

4 An honest comparison

- **Novel-view image quality: neural methods win decisively. They optimise exactly that objective**, so the result is unsurprising rather than mysterious — and it is worth stating that way, because it sets the correct expectation for the other rows.
- **Mesh quality for downstream geometric use: classical multi-view stereo remains competitive and is frequently better. A radiance field is not a surface (§2)**, and a mesh extracted from one inherits the density field's ambiguity at every thin or transparent structure.
- **Robustness to few views or badly distributed views: classical wins. Multi-view stereo degrades to a sparse or incomplete reconstruction, which is honest; neural methods degrade to confident fabrication (§2's floaters and invented back sides)**, which is not. **That difference in failure mode matters more than the average-case quality gap** for any system that must know when it does not know.
- **Metric accuracy: both depend on the same COLMAP poses, so neither escapes Module 01's scale theorem** — and a neural reconstruction is no more metric than the structure from motion that initialised it.
- **And for a game engine: splatting is the one you can actually render** at interactive rates on a consumer GPU, which is why this track cares and why it moved into production tools within two years of publication.
- **So choose by what the output is for: an image, a mesh, or a measurement. They are different deliverables, and the methods rank differently on each** — which is the same discipline as Module 09 §4's metric choice, applied to reconstruction.

WHY THIS MODULE IS THE CENTRE OF THE TRACK

CSCE 641 gave the rasterisation pipeline. CSCE 647 gave the rendering integral, the volumetric transport, and the sampling theory. CSCE 645 gave the surface representations. CSCE 669 gave the optimiser and the convergence theory.

This module is all four at once, pointed backwards: a renderer, differentiated, optimised against photographs, producing a scene. **Nothing in it would be approachable without those four, and with them it is almost straightforward.**

Which is why Graphics and Vision are one track and not two. The forward model is the thing you invert, and you cannot invert what you do not understand — which was Module 01's opening claim, and this module is the evidence for it.

And it is the clearest case in this program of a subject that could only have been built by people who knew both halves. The method is not a vision advance that happens to use rendering, nor a graphics advance that happens to use photographs; **it is the composition, and it was unavailable to anyone holding only one side.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Mildenhall et al. — NeRF: Representing Scenes as Neural Radiance Fields (free)	The §1 and §2 method. Readable, with a reference implementation and the positional-encoding ablation that makes §2's point.
Kerbl et al. — 3D Gaussian Splatting for Real-Time Radiance Field Rendering (free)	The §3 method , with the adaptive density control described properly.
Müller et al. — Instant Neural Graphics Primitives (free)	The multiresolution hash encoding that made radiance fields train in seconds — and a good example of a data-structure contribution beating an architectural one.
Tewari et al. — Advances in Neural Rendering (free survey)	The field laid out systematically , including the surface-versus-volume question of §2.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement volume rendering** along a ray for a hand-built density field, and verify the transmittance integral.
- 2 **Train a small NeRF on your own captured scene**, using COLMAP poses.
- 3 **Train it without positional encoding** and compare. Report the frequency content of each result.
- 4 **Perturb the camera poses by one degree** and report the degradation.
- 5 **Capture a scene from one side only** and render the unseen side. Report what was invented.
- 6 **Capture a scene containing a mirror** and inspect the recovered geometry behind it.
- 7 **Train Gaussian splatting on the same scene** and compare training time, render speed, and quality.
- 8 **Run COLMAP's dense multi-view stereo on the same images** and compare the meshes.
- 9 **Compare all three on a deliberately sparse capture** of ten images, and describe how each fails.
- 10 **Import a splat into a renderer** and report the achieved frame rate on your GPU.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 How does neural reconstruction avoid the correspondence problem?
- 2 Write the volume rendering integral and say what transmittance is.
- 3 Why is a volume used rather than a surface?
- 4 Why is positional encoding necessary, and what theory explains it?

- 5 Name five NeRF failure modes and relate each to Module 01.
- 6 Why is the mirror failure particularly instructive?
- 7 Why is Gaussian splatting fast, in three specific respects?
- 8 What is adaptive density control and what does it replace?
- 9 Compare neural and classical reconstruction on five axes.